

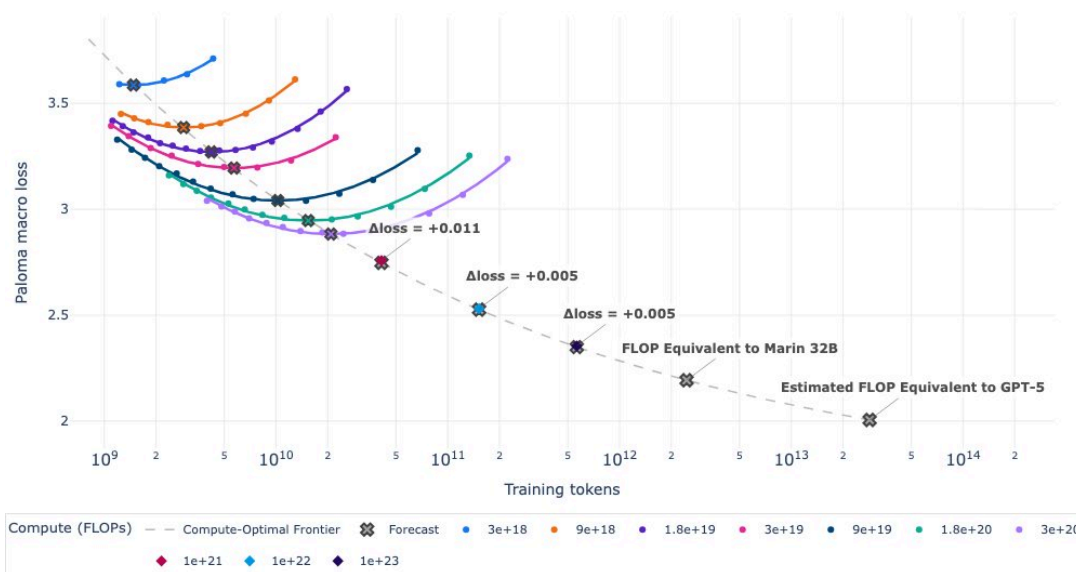
lecture_02.py



```

1 import math
2 import torch.nn.functional as F
3 import timeit
4 from typing import Iterable
5 import torch
6 from torch import nn
7 from einops import rearrange, einsum, reduce
8
9 from edtrace import text, image, link
10 from lecture_util import article_link
11 from gpu_util import cuda_if_available, get_max_memory_usage
12 from facts import h100_flop_per_sec, h100_bytes_per_sec
13 from references import deepseek_v3_2_2025, adagrad_2011, nemotron_3_super_2026
14
15
16 def main():
17     Announcements:
18     • Join the CS336 slack
19     • Sign up on Modal with your Stanford email
20     • Read the AI policy guide
21     • Read the cluster guide
22
23     Marin 1e23 FLOPs run finished and matched forecasts!
24 
```

Delphi Scaling Suite Results



```

25
26 Last lecture: overview, tokenization
27 Today: resource accounting (systems)
28
29 Recall: what's the best model one can train given fixed resources (compute, memory)?
30 In other words: maximize (computational) efficiency.
31 Prerequisite: understand the resources (compute, memory) for a given computation.
32

```

```

33 motivating_questions()
34

```

```

35 What knowledge to take away from this lecture:

```

- Mechanics: straightforward (PyTorch semantics)
- Mindset: resource accounting (remember to do it)
- Intuitions: get a sense of how resources are spent, no ML magic today

```

39
40 # Memory accounting

```

```

41  tensors_basics()
42  tensors_memory()
43  tensors_on_gpus()
44
45  # Compute accounting
46  tensor_einops()
47  tensor_operations_flops()
48
49  arithmetic_intensity()
50
51  # Memory and compute accounting for training
52  deep_network()
53  gradients_basics()
54  gradients_flops()
55  optimizer()
56  train_loop()
57
58  # More memory optimizations
59  gradient_accumulation()
60  activation_checkpointing()
61
62  Summary:
63  • Everything is operations on tensors (parameters, gradients, activations, optimizer states, data)
64  • einops: better way to think about tensor operations
65  • 6 (# data points) (# parameters) FLOPs per training step
66  • Arithmetic intensity / roofline analysis: compute-bound or memory-bound?
67  • Matrix multiplications are compute-bound, elementwise operations are memory-bound
68  • Gradient accumulation, activation checkpointing: reduce memory to use bigger batch sizes
69
70
71  def motivating_questions():
72      Question: How long would it take to train a 70B parameter model on 15T tokens on 1024 H100s?
73      total_flops = 6 * 70e9 * 15e12
74      h100_flop_per_sec = 1979e12 / 2
75      mfu = 0.5
76      flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24
77      days = total_flops / flops_per_day
78
79      Question: What's the largest model that can you can train on 8 H100s using AdamW?
80      h100_bytes = 80e9
81      bytes_per_parameter = 2 + 2 + (4 + 4) # parameters (2), gradients (2), optimizer state (4 + 4)
82      num_parameters = (h100_bytes * 8) / bytes_per_parameter
83      Caveat: activations are not accounted for (depends on batch size and sequence length), so this is an upper
      bound.
84
85      This is a rough back-of-the-envelope calculation.
86      But it gives you the flavor of napkin math one can quickly do to get a sense of resources.
87
88
89  def tensors_basics():
90      Tensors are the basic building block for storing everything:
91      • data
92      • parameters
93      • gradients
94      • optimizer state
95      • activations
96
97      Example: parameters of the DeepSeek v3.2 model [DeepSeek-AI+ 2025]
98      [DeepSeek v3.2 model on Hugging Face]
99
100     Each tensor has a rank, which is the number of dimensions.
101     x = torch.zeros(4)          # rank 1 tensor (vector)
102     x = torch.zeros(4, 8)       # rank 2 tensor (matrix)
103     x = torch.zeros(4, 8, 2)    # rank 3 tensor

```

In Transformers, will see tensors of rank 4:

$B = 32$ # Batch size

$S = 16$ # Sequence length

$H = 16$ # Number of heads

$D = 64$ # Hidden dimension per head

$x = \text{torch.zeros}(B, S, H, D)$

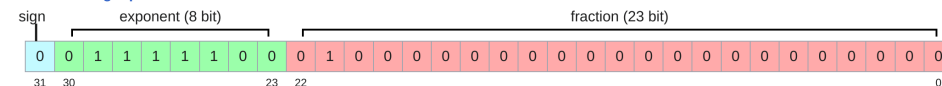
```
def tensors_memory():
```

Elements of tensors are generally floating point numbers.

fp32

[\[Wikipedia\]](#)

IEEE 754 single-precision 32-bit float



The fp32 data type (also known as float32 or single precision) is the default.

Traditionally, in scientific computing, fp32 is the baseline; you could use double precision (fp64) in some cases.

In deep learning, you can be a lot sloppier.

Let's examine memory usage of these tensors.

Memory is determined by the (i) number of values and (ii) data type of each value.

```
x = torch.zeros(4, 8)
```

```
assert x.dtype == torch.float32 # Default type
```

```
assert x.numel() == 4 * 8
```

```
assert x.element_size() == 4 # Float is 4 bytes
```

```
assert get_memory_usage(x) == 4 * 8 * 4 # 128 bytes
```

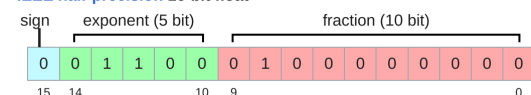
One matrix in the feedforward layer of GPT-3:

```
assert get_memory_usage(torch.empty(12288 * 4, 12288)) == 2304 * 1024 * 1024 # 2.3 GB
```

fp16

[\[Wikipedia\]](#)

IEEE half-precision 16-bit float



The fp16 data type (also known as float16 or half precision) cuts down the memory.

```
x = torch.zeros(4, 8, dtype=torch.float16)
```

```
assert x.element_size() == 2
```

However, the dynamic range (especially for small numbers) isn't great.

```
x = torch.tensor([1e-8], dtype=torch.float16)
```

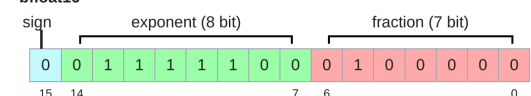
```
assert x == 0 # Underflow!
```

If this happens when you train, you can get instability.

bf16

[\[Wikipedia\]](#)

bf16



Google Brain developed brain floating point (bf16) in 2018 to address this issue.

bf16 uses the same memory as fp16 but has the same dynamic range as fp32!

The only catch is that the resolution is worse, but this matters less for deep learning.

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
```

```
assert x != 0 # No underflow!
```

Mixed precision

Implications on training:

- Training with fp32 works, but requires lots of memory.
- Training with fp16 and even bf16 is risky, and you can get instability.

Solution: mixed precision training [\[Micikevicius+ 2017\]](#)

- Use bf16 for parameters, activations, and gradients
- Use fp32 for optimizer states

Pytorch has an automatic mixed precision (AMP) library. [\[docs\]](#)

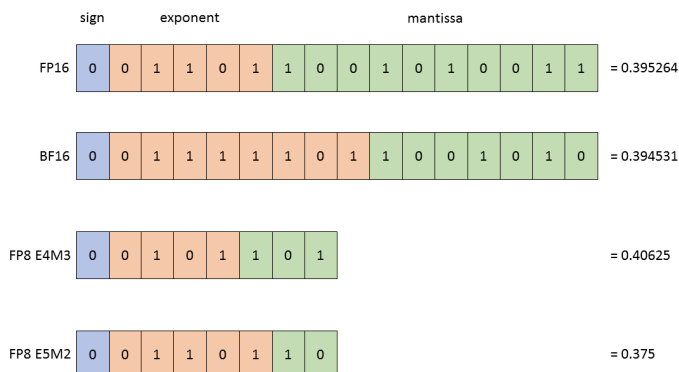
Tries to cast things into bf16 when safe (matmuls, not exp).

```
with torch.amp.autocast("cuda", dtype=torch.bfloat16):
```

```
    x = torch.zeros(4, 8)
```

fp8

In 2022, fp8 was standardized, motivated by machine learning workloads [primer](#).



H100s support two variants of FP8: E4M3 (range [-448, 448]) and E5M2 ([-57344, 57344]).

Reference: [\[Micikevicius+ 2022\]](#)

fp4

In 2025, NVIDIA developed [nvfp4](#)

Only 4 bits per value!

Values: -6, -4, -3, -2, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2, 3, 4, 6

Use a separate scale factor per block, so actually get more dynamic range (but just can't vary freely from neighbors).

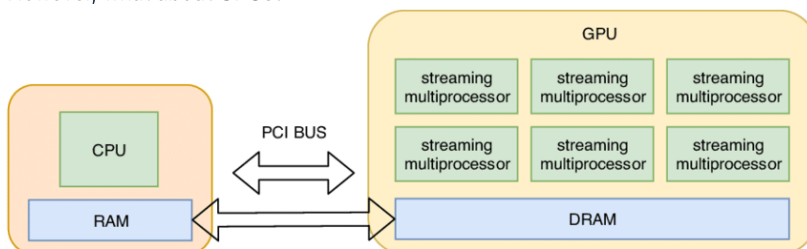
Nemotron 3 Super was trained in NVFP4

[\[Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning\]](#)

Some of this is done in NVIDIA libraries outside of user control.

```
def tensors_on_gpus():
    By default, tensors are stored in CPU memory.
    x = torch.zeros(32, 32)
    assert x.device == torch.device("cpu")
```

However, what about GPUs?



```
device = cuda_if_available()
```

In order to take advantage of the massive parallelism of GPUs, we need to move them to GPU memory.

```
x = x.to(device)
```

```

195
196 Or create the tensor directly on the GPU:
197 with torch.device(device):
198     x = torch.zeros(32, 32)
199     assert x.device == device
200
201
202 def tensor_einops():
203     einops_motivation()
204
205 Einops is a library for manipulating tensors where dimensions are named.
206 It is inspired by Einstein summation notation (Einstein, 1916).
207 \[Einops tutorial\]
208
209 einops_einsum()
210 einops_reduce()
211 einops_rearrange()
212
213
214 def einops_motivation():
215     Traditional PyTorch code:
216     x = torch.ones(2, 2, 3) # batch seq hidden
217     y = torch.ones(2, 2, 3) # batch seq hidden
218     z = x @ y.transpose(-2, -1) # batch seq seq
219     Easy to mess up the dimensions (what is -2, -1?)...
220
221
222 def einops_einsum():
223     Einsum is generalized matrix multiplication with good bookkeeping.
224
225     x = torch.ones(3, 4) # seq1 hidden
226     y = torch.ones(4, 3) # hidden seq2
227
228     # Old way
229     z = x @ y # seq1 seq2
230
231     # New (einops) way
232     z = einsum(x, y, "seq1 hidden, hidden seq2 -> seq1 seq2")
233
234     Let's try a more complex example...
235
236     x = torch.ones(2, 3, 4) # batch seq1 hidden
237     y = torch.ones(2, 3, 4) # batch seq2 hidden
238
239     # Old way
240     z = x @ y.transpose(-2, -1) # batch seq1 seq2
241
242     # New (einops) way
243     z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
244     Dimensions that are not named in the output are summed over.
245
246     # Or can use `...` to represent broadcasting over any number of dimensions
247     z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
248
249
250 def einops_reduce():
251     You can reduce a single tensor via some operation (e.g., sum, mean, max, min).
252     x = torch.ones(2, 3, 4) # batch seq hidden
253
254     # Old way
255     y = x.sum(dim=-1)
256
257     # New (einops) way
258     y = reduce(x, "... hidden -> ...", "sum")

```

```

259
260
261 def einops_rearrange():
262     Sometimes, a dimension represents two dimensions
263     ...and you want to operate on one of them.
264
265     x = torch.ones(3, 8) # seq total_hidden
266     ...where total_hidden is a flattened representation of heads * hidden1
267     w = torch.ones(4, 4) # hidden1 hidden2
268
269     # Break up `total_hidden` into two dimensions (`heads` and `hidden1`)
270     x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
271
272     # Perform the transformation by `w`
273     x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
274
275     # Combine `heads` and `hidden2` back together
276     x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")
277
278
279 def tensor_operations_flops():
280     Having gone through all the operations, let us examine their computational cost.
281
282     A floating-point operation (FLOP) is a basic operation like addition (x + y) or multiplication (x y).
283
284     Two terribly confusing acronyms (pronounced the same!):
285     • FLOPs: floating-point operations (measure of computation done)
286     • FLOP/s: floating-point operations per second (also written as FLOPS), which is used to measure the speed
      of hardware.

```

Intuitions

```

289 Training GPT-3 (2020) took 3.14e23 FLOPs. \[article\]
290 Training GPT-4 (2023) is speculated to take 2e25 FLOPs. \[article\]
291
292 H100 has a peak performance of 1979 teraFLOP/s with sparsity, 50% without \[spec\]
293 h100_flop_per_sec = 1979e12 / 2
294
295 8 H100s for 2 weeks:
296 total_flops = 8 * 2 * (60 * 60 * 24 * 7) * h100_flop_per_sec
297
298

```

Linear model

```

299 if torch.cuda.is_available():
300     B = 16384 # Number of points
301     D = 32768 # Dimension of each point
302     K = 8192 # Number of outputs
303 else:
304     B = 1024
305     D = 256
306     K = 64
307
308 x = torch.ones(B, D, device=cuda_if_available())
309 w = torch.randn(D, K, device=cuda_if_available())
310 y = x @ w
311
312 How many FLOPs is this matmul?
313 We have one multiplication (x[i][j] * w[j][k]) and one addition per (i, j, k) triple.
314 actual_num_flops = 2 * B * D * K
315
316 We can also time this operation to see how long it takes.
317 actual_time = benchmark(lambda: x @ w)
318
319 The actual FLOP/s of this operation:
320 actual_flop_per_sec = actual_num_flops / actual_time

```

Each GPU has a specification sheet that provides the peak performance.

- Example: [\[H100 spec\]](#)

Note that the FLOP/s depends heavily on the data type!

```
promised_flop_per_sec = get_promised_flop_per_sec(x.dtype)
```

Model FLOPs utilization (MFU)

Definition: $MFU = (\text{actual FLOP/s}) / (\text{promised FLOP/s})$ [ignore communication/overhead]

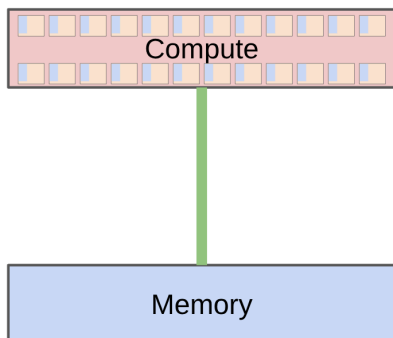
```
mfu = actual_flop_per_sec / promised_flop_per_sec if promised_flop_per_sec else None
```

Usually, MFU of ≥ 0.5 is quite good!

But why is MFU not closer to 1?

To answer this question, we need to look more closely at how computations are done on GPUs...

```
def arithmetic_intensity():
```



How to compute a thing:

1. Send inputs from memory to accelerator
2. Perform computation
3. Send outputs from accelerator to memory

How long does this take?

Depends on two things:

1. Accelerator speed (FLOP/s)
2. Memory bandwidth (bytes/s)

```
assert h100_flop_per_sec == 1979e12 / 2 # Half without sparsity
```

```
assert h100_bytes_per_sec == 3.35e12
```

```
arithmetic_intensity_relu()
```

```
arithmetic_intensity_gelu()
```

```
arithmetic_intensity_dot_product()
```

```
arithmetic_intensity_matrix_vector_product()
```

```
arithmetic_intensity_matmul()
```

```
# Let's visualize it
```

```
roofline_plots()
```

```
def arithmetic_intensity_relu():
```

```
    n = 1024 * 1024
```

```
    x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
```

```
    y = torch.relu(x)
```

```
    bytes = (2 * n) + (2 * n) # Read x, write y (bf16 is 2 bytes/float)
```

```
    flops = n # n comparisons
```

```
    communication_time = bytes / h100_bytes_per_sec
```

```
    computation_time = flops / h100_flop_per_sec
```

```

374 Assume we can overlap communication and computation perfectly.
375 total_time = max(communication_time, computation_time)
376
377 What is the bottleneck?
378 • Memory-bound: communication time > computation time
379 • Compute-bound: computation time > communication time
380
381 In this case, ReLU is memory-bound.
382
383 Alternative way to see this:
384 Accelerator intensity: how much work can the accelerator do per byte transferred?
385 h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
386
387 Arithmetic intensity: how much actual work per byte for this workload?
388 arithmetic_intensity = flops / bytes # ~1/4
389
390 What is the bottleneck?
391 • Memory-bound: arithmetic intensity < accelerator intensity
392 • Compute-bound: arithmetic intensity > accelerator intensity
393
394 assert arithmetic_intensity < h100_accelerator_intensity
395
396 In general, we'll find ourselves memory bound.
397 Can we increase arithmetic intensity?
398
399
400 def arithmetic_intensity_gelu():
401     n = 1024 * 1024
402     x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
403     y = F.gelu(x) # GELU(x) = 0.5 x (1 + tanh(sqrt(2/pi) (x + 0.044715 x^3)))
404
405     bytes = (2 * n) + (2 * n) # Read x, write y (bf16 is 2 bytes/float)
406     flops = 20 * n # tanh can be approximated in various ways (e.g., polynomials)
407
408     arithmetic_intensity = flops / bytes
409
410     h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
411     assert arithmetic_intensity < h100_accelerator_intensity
412
413 Note that GeLU does more work than ReLU per byte moved, so it has higher arithmetic intensity.
414 But still memory-bound!
415 In other words, ReLU is not faster than GeLU (when doing things in an isolated way).
416
417
418 def arithmetic_intensity_dot_product():
419     n = 1024 * 1024
420     x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
421     w = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
422     y = x @ w
423
424     bytes = (2 * n) + (2 * n) + 2 # Read x, read w, write y
425     flops = 2 * n - 1 # n multiplications, n-1 additions
426
427     arithmetic_intensity = flops / bytes # ~1/2
428
429     h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
430     assert arithmetic_intensity < h100_accelerator_intensity
431     Memory-bound!
432
433
434 def arithmetic_intensity_matrix_vector_product():
435     n = 1024
436     x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
437     w = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())

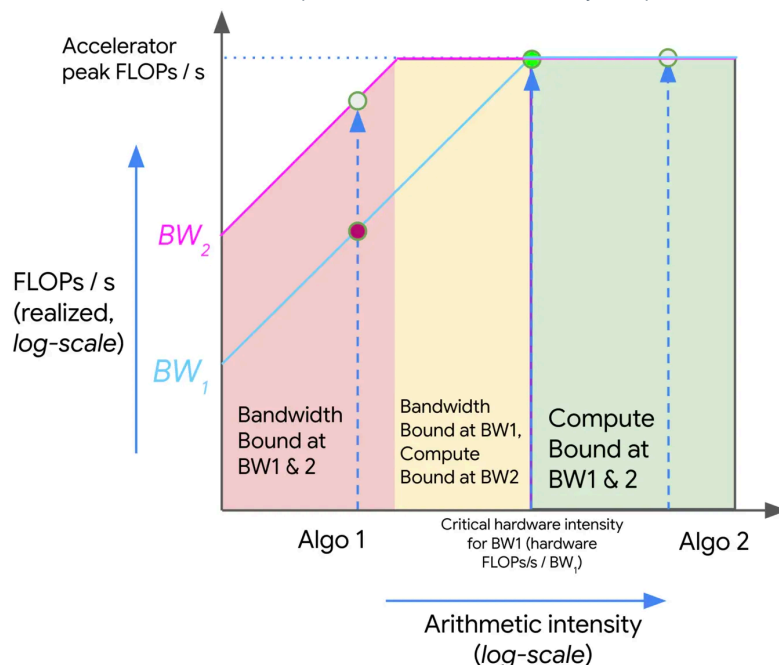
```



```

438     y = x @ w
439
440     bytes = (2 * n) + (2 * n * n) + (2 * n) # Read x, read w, write y
441     flops = n * (2 * n - 1) # n dot-products
442
443     arithmetic_intensity = flops / bytes # ~1
444
445     h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
446     assert arithmetic_intensity < h100_accelerator_intensity
447     Memory-bound!
448
449 def arithmetic_intensity_matmul():
450     n = 1024
451     x = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
452     w = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
453     y = x @ w
454
455     bytes = (2 * n * n) + (2 * n * n) + (2 * n * n) # Read x, read w, write y
456     flops = n * n * (2 * n - 1) # n^2 dot products
457
458     arithmetic_intensity = flops / bytes # ~n/3
459
460     h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
461     assert arithmetic_intensity > h100_accelerator_intensity
462     Finally, compute-bound!
463
464     As long as we have large matrices, we're compute-bound (saturating the accelerator).
465     Training Transformers involves big matrix multiplications.
466     Matrix-vector product is what happens during inference, which is why inference is memory-bound.
467
468     Note: arithmetic/accelerator intensity also depends on the precision (bf16 versus fp32).
469
470
471 def roofline_plots():
472     We can visualize the relationship between arithmetic intensity and performance using roofline plots.
473

```



- Each slice on the x-axis is a particular computation (with some arithmetic intensity)
- Each piecewise linear function corresponds to a particular hardware
- Kink is the accelerator intensity (transition from memory-bound to compute-bound)

We can now relate this back to MFU:

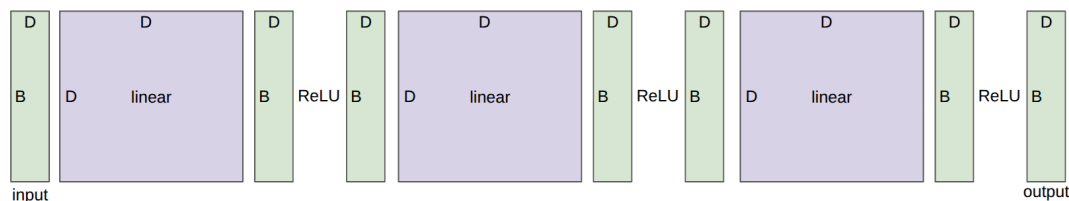
$$\text{MFU} = \min(1, \text{arithmetic-intensity} / \text{accelerator-intensity})$$

[reference]

```

482
483
484 def gradients_basics():
485     So far, we've constructed tensors and passed them through operations (forward).
486     Now, we're going to compute the gradient (backward).
487
488     As a simple example, let's consider the simple linear model:
489      $y = 0.5 (x * w - 5)^2$ 
490
491     Forward pass: compute loss
492     x = torch.tensor([1., 2, 3])
493     w = torch.tensor([1., 1, 1], requires_grad=True) # Want gradient
494     pred_y = x @ w
495     loss = 0.5 * (pred_y - 5).pow(2)
496
497     Backward pass: compute gradients
498     loss.backward()
499     assert torch.equal(w.grad, torch.tensor([1, 2, 3]))
500
501
502 def gradients_flops():
503     Let us count the FLOPs for computing gradients.
504
505

```



```

506
507 B = 1024 # Number of points
508 D = 256 # Dimension
509
510 Define a simplified model (2-layer linear network):
511 x = torch.ones(B, D, device=cuda_if_available())
512 w1 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)
513 w2 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)
514
515 # Forward pass
516 h1 = einsum(x, w1, "batch in, in out -> batch out") # x
517 h2 = einsum(h1, w2, "batch in, in out -> batch out") # h1
518 loss = (h2.mean() - 0)**2 # Regress everything to 0 (arbitrary)
519
520 # Backward pass
521 h1.retain_grad() # For debugging
522 h2.retain_grad() # For debugging
523 loss.backward()
524

```

Zoom in on one layer

Let's focus on the second layer ($h2 = h1 @ w2$)

Forward pass: Recall the number of forward FLOPs:

```
num_forward_flops = 2 * B * D * D
```

Backward pass: How many FLOPs is running the backward pass?

We need to compute:

- $h1.grad = d \text{ loss} / d h1$
- $w2.grad = d \text{ loss} / d w2$

```

536
537 h1_grad = einsum(h2.grad, w2, "batch out, in out -> batch in")
538 assert torch.allclose(h1.grad, h1_grad)
539

```

```

540 w2_grad = einsum(h2.grad, h1, "batch out, batch in -> in out")
541 assert torch.allclose(w2.grad, w2_grad)
542
543 num_backward_flops = (2 * B * D * D) + (2 * B * D * D)
544

```

Note that the backward pass is 2x more expensive than the forward pass.

Consider all layers

This was just for w_2 , need to apply it to all parameters in the network.

Putting it together:

- Forward pass: 2 (# data points) (# parameters) FLOPs
- Backward pass: 4 (# data points) (# parameters) FLOPs
- Total: 6 (# data points) (# parameters) FLOPs

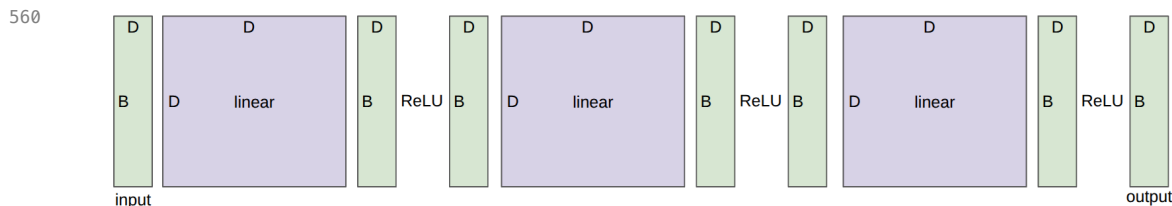
This is for multilayer perceptrons (MLPs)

...but it turns out to be a good approximation for Transformers for short context lengths as well.

```

559 def deep_network():

```



Consider a deep network with L layers and D -dimensional inputs, activations, and outputs.

```

563 # Define the network
564 D = 8 # Dimensionality of input, activations, and output
565 L = 3 # Number of layers
566 model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
567
568 num_parameters = get_num_parameters(model)
569 assert num_parameters == (D * D) * L
570
571 # Run the model on a batch of data
572 B = 4 # Batch size
573 x = torch.randn(B, D, device=cuda_if_available())
574 y = model(x)
575
576
577 class Block(nn.Module):
578     """Simple block that applies a linear transformation followed by a ReLU nonlinearity."""
579     def __init__(self, dim: int):
580         super().__init__()
581         self.weight = nn.Parameter(torch.randn(dim, dim) / math.sqrt(dim))
582
583     def forward(self, x: torch.Tensor) -> torch.Tensor:
584         x = x @ self.weight # Linear
585         x = F.relu(x) # Activation
586         return x
587
588
589 class DeepNetwork(nn.Module):
590     """Map `dim`-vector to a `dim`-vector."""
591     def __init__(self, dim: int, num_layers: int):
592         super().__init__()
593         self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])
594
595     def forward(self, x: torch.Tensor) -> torch.Tensor:
596         # Apply all the layers sequentially
597         for layer in self.layers:

```

```

598         x = layer(x)
599     return x
600
601
602 def optimizer():
603     Recall our deep network.
604     B = 2 # Batch size
605     D = 4 # Dimensionality of input, activations, and output
606     L = 3 # Number of layers
607     model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
608
609     Let's define the AdaGrad optimizer
610     • momentum = SGD + exponential averaging of grad
611     • AdaGrad = SGD + averaging by grad^2
612     • RMSProp = AdaGrad but with exponential averaging of grad^2
613     • Adam = RMSProp + momentum
614
615     AdaGrad [Duchi+ 2011]
616     optimizer = AdaGrad(model.parameters(), lr=0.01)
617     state = model.state_dict()
618
619     # Compute gradients
620     x = torch.randn(B, D, device=cuda_if_available())
621     y = torch.tensor([4., 5.], device=cuda_if_available())
622     pred_y = model(x).mean()
623     loss = F.mse_loss(input=pred_y, target=y)
624     loss.backward()
625
626     # Take a step
627     optimizer.step()
628     optimizer_state = {i: dict(p_state) for i, (p, p_state) in enumerate(optimizer.state.items())}
629
630     # Free up the memory
631     optimizer.zero_grad(set_to_none=True)
632
633

```

Memory

```

634
635 num_parameters = D * D * L
636 parameter_memory = 2 * num_parameters # (2 bytes for bf16)
637 gradient_memory = 2 * num_parameters # (2 bytes for bf16)
638 optimizer_state_memory = 4 * num_parameters # (4 bytes for fp32)
639 activation_memory = 2 * (B * D * L) # (2 bytes for bf16)
640 It is customary to use fp32 for stability (accumulating averages over powers over many steps).
641 Optimizer state memory:
642 • AdaGrad: 4 bytes/parameter for storing second moments
643 • Adam: 8 bytes/parameter for storing first and second moments
644
645 # Putting it all together
646 total_memory = parameter_memory + activation_memory + gradient_memory + optimizer_state_memory
647

```

Compute (for one training step)

```

648
649 num_parameters = D * D * L
650 flops = 6 * B * num_parameters
651

```

Transformers

653 The accounting for a Transformer is more complicated, but the same idea.
 654 Assignment 1 will ask you to do that.

656 Blog post describing memory usage for Transformer training [\[article\]](#)
 657 Blog post describing FLOPs for a Transformer: [\[article\]](#)

```

658
659
660 class AdaGrad(torch.optim.Optimizer):

```

```

661     def __init__(self, params: Iterable[nn.Parameter], lr: float = 0.01):
662         super(AdaGrad, self).__init__(params, dict(lr=lr))
663
664     def step(self):
665         for group in self.param_groups:
666             lr = group["lr"]
667             for p in group["params"]:
668                 # Optimizer state
669                 state = self.state[p]
670                 grad = p.grad.data
671
672                 # Get squared gradients g2 = sum_{i<t} g_i^2
673                 g2 = state.get("g2", torch.zeros_like(grad))
674
675                 # Update optimizer state
676                 g2 += torch.square(grad)
677                 state["g2"] = g2
678
679                 # Update parameters
680                 p.data -= lr * grad / torch.sqrt(g2 + 1e-5)
681
682
683     def train_loop():
684         # True linear function with weights (0, 1, 2, ..., D-1)
685         D = 16 # Dimensionality
686         true_w = torch.arange(D, dtype=torch.float32, device=cuda_if_available())
687
688         # Data loader that generates (x, y) pairs
689         B = 4 # Batch size
690         def get_batch() -> tuple[torch.Tensor, torch.Tensor]:
691             x = torch.randn(B, D).to(cuda_if_available())
692             true_y = x @ true_w
693             return (x, true_y)
694
695         # Define the model and optimizer
696         L = 2 # Number of layers
697         model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
698         optimizer = AdaGrad(model.parameters(), lr=0.01)
699
700         # Train!
701         num_train_steps = 10
702         for t in range(num_train_steps):
703             # Get data
704             x, y = get_batch()
705
706             # Forward (compute loss)
707             pred_y = model(x).mean()
708             loss = F.mse_loss(pred_y, y)
709
710             # Backward (compute gradients)
711             loss.backward()
712
713             # Update parameters
714             optimizer.step()
715             optimizer.zero_grad(set_to_none=True)
716
717
718     def gradient_accumulation():
719         Large batch sizes: improve training stability
720         However, activation memory scales with batch size, so might run out.
721         B = 64 # Batch size
722         D = 1024 # Dimensionality
723         L = 16 # Number of layers
724         activation_memory = 2 * B * D * L # (2 bytes for bf16)

```

- Compute gradient on micro batches
- Accumulate the gradients (don't zero it out)
- Every batch_size / micro_batch_size steps, update the parameters and zero out the gradients

```
micro_batch_size = 256
activation_memory = 2 * micro_batch_size * D * L # (2 bytes for bf16)
```

The diagram illustrates a neural network architecture with three layers. Each layer is composed of three main components: a green block labeled 'D', a purple block labeled 'linear', and another green block labeled 'B'. The layers are connected sequentially, with 'ReLU' activation functions placed between the 'B' blocks of adjacent layers. The input is labeled 'input' and the output is labeled 'output'.

```

785         # KEY: only store activations at checkpoints, recompute the rest
786         x = torch.utils.checkpoint.checkpoint(layer, x)
787     return x
788
789 #####
790
791 def get_memory_usage(x: torch.Tensor):
792     return x.numel() * x.element_size()
793
794
795 def get_promised_flop_per_sec(dtype: torch.dtype) -> float:
796     """Return the peak FLOP/s for `device` operating on `dtype`."""
797     if not torch.cuda.is_available():
798         # No CUDA device available, so can't get FLOP/s
799         return 1
800     properties = torch.cuda.get_device_properties(cuda_if_available())
801
802     if "A100" in properties.name:
803         # https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-
1758950-r4-web.pdf
804         if dtype == torch.float32:
805             return 19.5e12
806         if dtype in (torch.bfloat16, torch.float16):
807             return 312e12
808         raise ValueError(f"Unknown dtype: {dtype}")
809
810     if "H100" in properties.name:
811         # https://www.nvidia.com/en-us/data-center/h100/
812         if dtype == torch.float32:
813             return 67.5e12
814         if dtype in (torch.bfloat16, torch.float16):
815             return 1979e12 / 2 # 1979 is for sparse, dense is half of that
816         raise ValueError(f"Unknown dtype: {dtype}")
817
818     if "B200" in properties.name:
819         # https://www.primeline-solutions.com/media/categories/server/nach-gpu/nvidia-hgx-h200/nvidia-blackwell-b200-
datasheet.pdf
820         if dtype == torch.float32:
821             return 75e12
822         if dtype in (torch.bfloat16, torch.float16):
823             return 4.5e15 / 2 # 4.5e15 is for sparse, dense is half of that
824         raise ValueError(f"Unknown dtype: {dtype}")
825
826     # Unknown GPU: return None so caller can handle gracefully
827     return None
828
829
830 def benchmark(func, num_trials: int = 5) -> float:
831     """Return the number of seconds required to perform `func`."""
832
833     # Wait until previous CUDA threads are done
834     if torch.cuda.is_available():
835         torch.cuda.synchronize()
836
837     def run():
838         # Perform the operation
839         func()
840
841     # Wait until CUDA threads are done
842     if torch.cuda.is_available():
843         torch.cuda.synchronize()
844
845     # Time the operation `num_trials` times
846     total_time = timeit.timeit(run, number=num_trials)

```

```
847
848     return total_time / num_trials
849
850
851 def get_num_parameters(model: nn.Module) -> int:
852     return sum(param.numel() for param in model.parameters())
853
854
855 if __name__ == "__main__":
856     main()
```